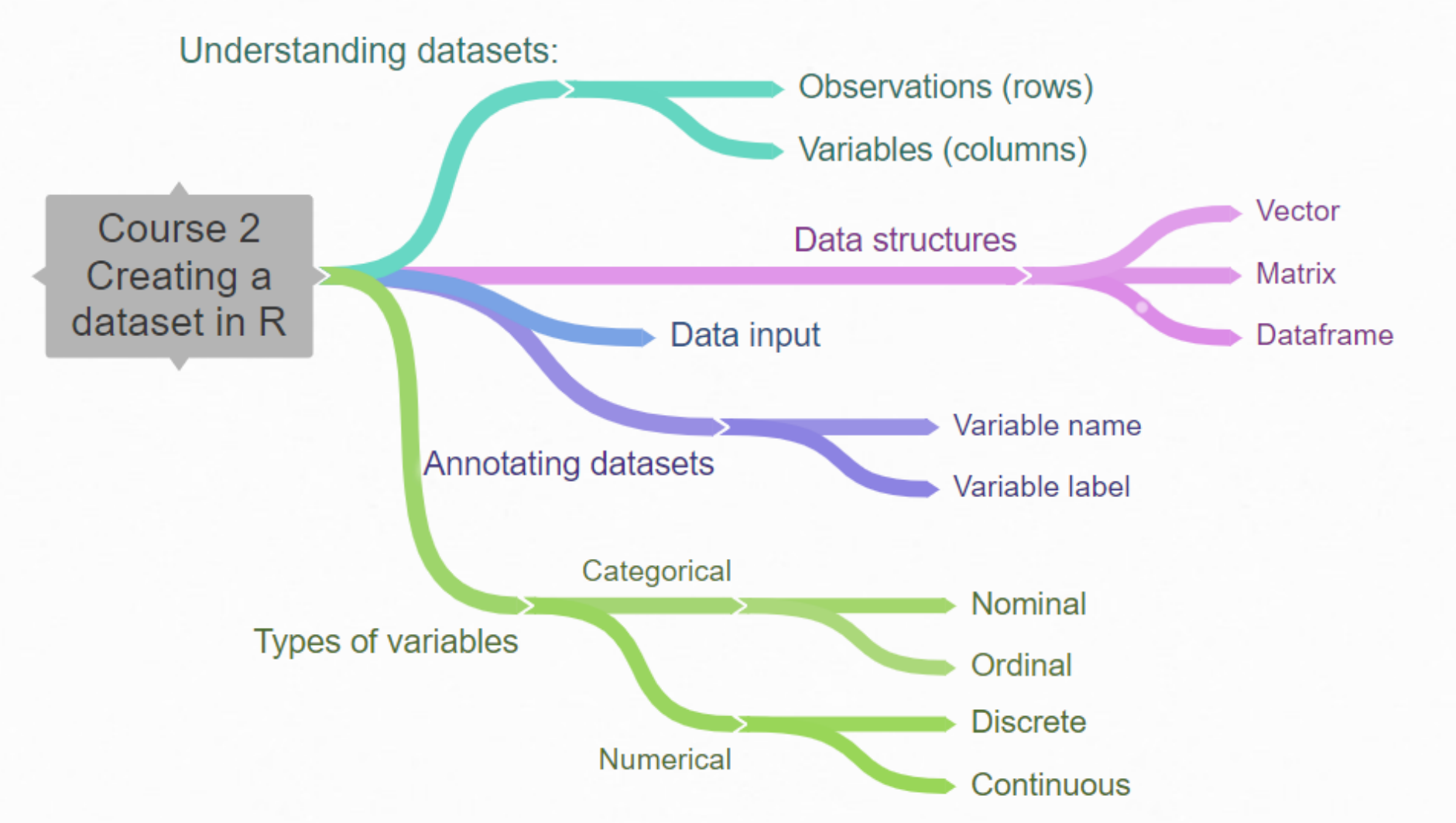
Course 3:
Basic data management in R (I)

Review


Course 2
Creating a
dataset in R
Understanding datasets:
Observations (rows)
Variables (columns)
Data structures
Vector
Matrix
Dataframe
Data input
Annotating datasets
Variable name
Variable label
Types of variables
Categorical
Nominal
Ordinal
Numerical
Discrete
Continuous

1. Creating new variables
2. Recoding variables
3. Renaming variables
4. Missing values
5. Type conversions
6. Summary

Creating new variables

```
1        variable <- expression
```

Operator	Discription
+	加法
-	减法
*	乘法
/	除法
^or **	阶乘
x%%y	余数：e.g., 5%%2=1
x%/%y	整除：e.g., 5%/%2=2

1 Creating new variables

2 Recoding variables

3 Renaming variables

4 Missing values

5 Type conversions

6 Summary

Recoding variables

重编码涉及根据同一变量的现有值和/或其他变量的现有值创建新值，比如：

1. 将连续变量更改为一组类别
   e.g., 根据年龄分成老中青三个组
2. 根据分隔值创建 0/1 变量
   e.g., 根据成绩创建是否合格变量（pass/fail）
3. 用正确的值替换错误编码的值

Recoding variables

To recode data, you can use one or more of R's logical operators (逻辑运算符):

Operator	Description
<	小于
<=	小于等于
>	大于
>=	大于等于
==	等于
!=	不等于
!x	非 x
x|y	x 为真或者 y 为真
x & y	x 为真并且 y 为真
isTRUE(x)	判断 x 是否为真

注：逻辑运算符是返回 TRUE 或 FALSE 的表达式

Logical operators

==

== 是一个逻辑运算符，用来比较两个值是否相等。

- 当使用 == 比较两个值时，如果这两个值相等，那么表达式的结果是 TRUE（真），如果不相等，结果是 FALSE（假）
- == 用于比较操作，而不是赋值操作

举个简单例子：

```
4 == 4
#这会返回 TRUE, 因为4等于4
4 == 5
# 这会返回 FALSE, 因为4不等于5
#再举个例子:
x <- c(1, 2, 3)
x[x == 3] <- 4
# 以上操作将使得x变成 c(1, 2, 4)
```

Logical operators

& (and)

- 当使用 & 连接两个条件时，只有当这两个条件同时为真时，整个表达式的结果才是 TRUE
- 如果其中一个或两个条件为假，整个表达式的结果就是 FALSE

举个例子，想象你在一个游乐园，只有当身高超过 120 厘米并且年龄超过 10 岁时，才能玩一个游戏。这可以用 R 语言表达为：

```
1         height > 120 & age > 10
2         #如果一个孩子身高130且12岁，则以上返回TRUE
```

Logical operators

| (or)

- 当使用 | 连接两个条件时，只要至少有一个条件为真，整个表达式的结果就是 TRUE
- 只有当两个条件都不满足时，结果才是 FALSE

再举个例子，想象在选择晚餐，你的舍友想吃披萨或者汉堡，这可以用 R 语言表达为：

```
1         food == "Pizza" | food == "Burger"
```

如果选择了披萨，这个表达式就是 TRUE，因为至少有一个条件满足

逻辑与 & 和逻辑或 |

简单来说:

- & 要求所有条件都必须满足
- | 只要求至少满足一个条件

在进行数据筛选或条件判断时逻辑符号非常有用，比如筛选符合特定条件的数据行或者在编写更复杂的逻辑表达式时。

```
variable[condition] <- expression
# 比如:
x <- c(-1, 0.2, 0.5, 2, 3, 4, 20)
x[x > 0 & x < 1] <- 1
#再比如:
x[x == -1] <- 0
```

注：语句 variable[condition] <- expression 仅当条件为 TRUE 时才会进行赋值。

Logical operators

! 逻辑非运算符

! 用于反转一个逻辑表达式的真值

- 如果表达式为 TRUE，使用 ! 后会变成 FALSE
- 如果表达式为 FALSE，使用 ! 后会变成 TRUE

这个运算符在需要根据条件排除某些情况时非常有用。

Recoding variables

除了使用逻辑符号进行数据筛选及重编码，还可以使用 ifelse()

```
1    ifelse(test, yes, no)
2    #假设我们有一个表示温度值的向量，我们想要根据温度将其分类为“
         Cold”（低于0度）、“Temperate”（0到25度）和“Hot”（高于
         25度）
3    temp <- c(-5, 2, 20, 30)
4    temp_cat <- ifelse(temp < 0, "Cold",
5                       ifelse(temp <= 25, "Temperate", "Hot"))
```

- test：要评估的条件，如果该条件为真（TRUE），则选择 yes 的对应值；如果条件为假（FALSE），则选择 no 的对应值
- yes：当测试结果为真时返回的值
- no：当测试结果为假时返回的值

1 Creating new variables

2 Recoding variables

3 Renaming variables

4 Missing values

5 Type conversions

6 Summary

Renaming variables

在 R 语言中，变量重命名是数据处理的常见任务，可以通过多种方式实现，比如直接操作数据框的列名，或者使用 dplyr 包中的 rename() 函数。假设我们有一个数据框 df:

```r
# 直接修改列名
names(df) <- c("newname1", "newname2")
# 查看结果
print(df)
#或者，如果只想修改某个特定的列名：
names(df)[names(df) == "oldname1"] <- "newname1"
```

1 Creating new variables

2 Recoding variables

3 Renaming variables

4 Missing values

5 Type conversions

6 Summary

Missing values

- 在 R 中，缺失值由符号 NA（不可用）表示。与 SAS 等程序不同，R 使用相同的缺失值符号表示字符和数字数据
- R 提供了许多函数来识别包含缺失值的观测值，函数 is.na( ) 识别缺失值是否存在

is.na( )

is.na( ) 函数应用在一个对象上，返回一个相同大小的对象：

- 如果元素是一个缺失值，则该项被替换为 TRUE；
- 如果元素不是一个缺失值，则该项被替换为 FALSE。

Missing values

例子 1：应用于单个数值向量

假设我们有一个包含一些数值和缺失值的向量：

```
numbers <- c(1, 2, NA, 4, NA, 6)
```

我们想知道哪些位置是缺失的，可以使用 is.na() 来检查：

```
is.na(numbers)
```

这将返回一个逻辑向量：FALSE, FALSE, TRUE, FALSE, TRUE, FALSE，其中 TRUE 表示相应位置的值是 NA

Missing values

例子 2：应用于数据框

假设我们有一个简单的数据框，记录了一些人的年龄：

```
people <- data.frame(names = c("Alice", "Bob", "Charlie", "
    David"), ages = c(25, NA, 30, NA))
```

我们可能对哪些人的年龄是缺失的感兴趣。使用 is.na()：

```
is.na(people$ages)
```

Missing values

1. 缺失值是不可比较的。
   逻辑测试 myvar == NA 永远不会为 TRUE, 必须使用像 is.na() 这样的缺失值函数来识别 R 数据对象中的缺失值。
2. R 不会将无限或不可能的值表示为缺失值
   这与 SAS 等其他程序处理此类数据的方式不同，正无限和负无限分别由符号 Inf 和 -Inf 表示

Missing values

Recoding values to missing

通过重新赋值，让 R 知道一些特别值意味着缺失值：

```
1    Data$age[Data$age == 999] <- NA
2    #任何等于999的年龄值都会被改变为NA
```

确保在分析数据之前，任何缺失的数据都被正确地编码为缺失，否则结果没有意义。

Missing values

忽略缺失值：na.rm=TRUE

大多数数值函数都有一个 na.rm = TRUE 选项，该选项会在计算之前移除缺失值，并将函数应用于剩余的值。

```
# 假设我们有以下数值向量，包含缺失值
numbers <- c(1, 2, NA, 4, 5)
# 默认情况下，mean()会返回NA
# 使用na.rm=TRUE选项忽略缺失值
mean(numbers, na.rm = TRUE)
#会返回3
```

使用 na.rm=TRUE 是处理包含缺失值数据的常用方法，它可以使代码不被 NA 干扰，适用于 R 中的许多数值函数，如 sum()、min()、max() 等。

Missing values

移除缺失值：na.omit()

na.omit() 用来移除任何含有缺失值的样本（行）。

```
students <- data.frame(name = c("Bob", "Charlie"),
                       score = c(90, NA),
                       age = c(21, 22))
clean_students <- na.omit(students)
#clean_students将只包含Bob的数据
```

注：使用 na.omit() 是处理缺失数据的快速方法，但它可能导致大量数据被删除，尤其是在数据集中缺失值分布广泛时。因此，应该谨慎使用，并考虑数据缺失的原因和模式，以及是否有更好的方法来处理缺失值，比如填充缺失值或使用模型来估计缺失值（topic for the future）。

1 Creating new variables

2 Recoding variables

3 Renaming variables

4 Missing values

5 Type conversions

6 Summary

Type conversions

在 R 语言中，类型转换是指将数据从一种类型转换为另一种类型的过程

Test	Convert
is.numeric()	as.numeric()
is.character()	as.character()
is.vector()	as.vector()
is.matrix()	as.matrix()
is.data.frame()	as.data.frame()
is.factor()	as.factor()
is.logical()	as.logical()

Type conversions

转换为字符型 as.character()

```
num <- 42
char_num <- as.character(num)
char_num
#这会将数值42转换为字符型"42"
```

Type conversions

从字符型转换回数值型 as.numeric()

```r
char_num <- "3.14"
num <- as.numeric(char_num)
#这会将字符型的"3.14"转换为数值型的3.14
```

转换为因子型 as.factor()

因子是 R 中用来表示分类数据的数据类型。分类数据指的是可以按照类别进行分组的数据，比如性别（男、女）、评级（高、中、低）等。

- as.factor() 函数用于将其他类型的向量转换为因子类型。这对于数据分析和统计建模特别重要，因为它告诉 R 如何正确处理这些分类变量

Type conversions

as.factor()

比如有一个字符向量，代表一组人的血型：

```
blood_types <- c("A", "B", "O", "AB", "O", "B")
#要将这个字符向量转换为因子，可以使用as.factor()：
factor_blood_types <- as.factor(blood_types)
#这样，factor_blood_types就成为了一个因子，R会自动识别出有4个水平（A、B、O、AB），即4个不同的类别。
```

factor()

factor() 函数主要用于创建新的因子变量，能够指定因子的水平（levels）、标签（labels）、是否有序（ordered factor）等。

```
data <- c("low", "medium", "high", "medium", "low")
f <- factor(data, levels = c("low", "medium", "high"))
#levels命令指定了low作为参照水平
#试试将high指定为参照水平？
f <- factor(data, levels = c("high", "medium", "low"))
```

1 Creating new variables

2 Recoding variables

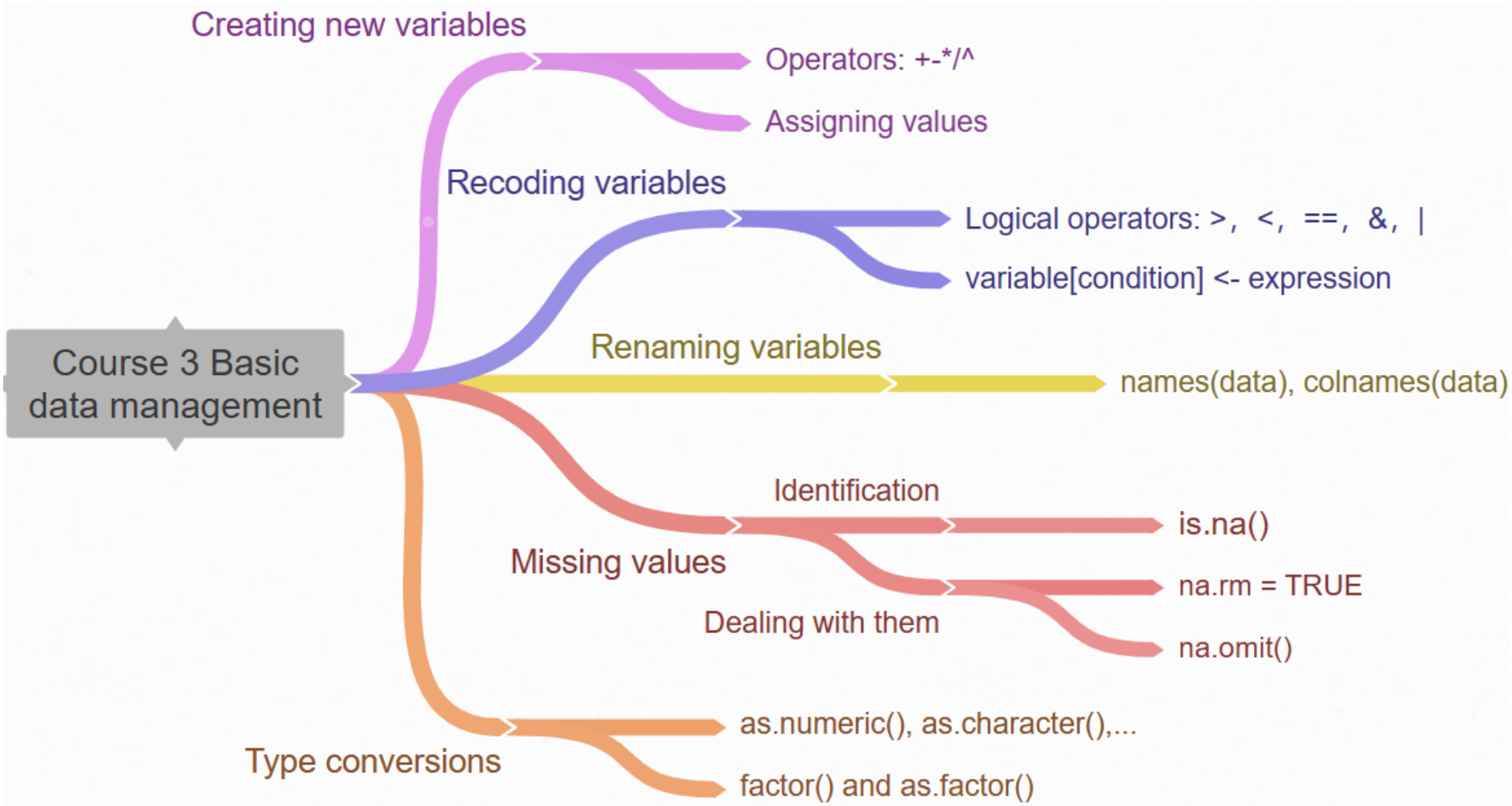
3 Renaming variables

4 Missing values

5 Type conversions

6 Summary

Summary


Course 3 Basic data management
Creating new variables
Operators: +-*/^
Assigning values
Recoding variables
Logical operators: >, <, ==, &, |
variable[condition] <- expression
Renaming variables
names(data), colnames(data)
Missing values
Identification
is.na()
Dealing with them
na.rm = TRUE
na.omit()
Type conversions
as.numeric(), as.character(),...
factor() and as.factor()